RKNN3 MCP Quick Start

Document ID: RK-JC-YF-440

Release Version: V0.1.0

Date: 2026-10-15

Confidentiality: □ Top Secret □ Secret □ Internal ■ Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This quick start guide covers connecting to RKNN3 MCP, starting deployment tasks, querying the knowledge base, and frequently asked questions. It helps users complete the initial setup and start using the service. RKNN3 MCP is currently in beta.

Intended Audience

This document is intended for RKNN3 MCP users who need model conversion, on-device deployment, or related capabilities.

Revision History

Version	Author	Date	Description	Reviewer
V0.1.0	AI Team	2026-10-15	Initial beta release, aligned with RKNN3 Toolkit V1.2.0	Vincent

Table of Contents

RKNN3 MCP Quick Start

1 Introduction to RKNN3 MCP

2 Quick Start

2.1 Before You Begin

2.2 Connecting to RKNN3 MCP

2.2.1 Connection Requirements

2.2.2 Connection Configuration

2.2.3 Verifying the Connection

2.3 Deployment Examples

2.3.1 Basic Example

2.3.2 Specifying Deployment Stages

2.3.3 Specifying Features

2.3.4 Minimal Deployment Prompt

3 Knowledge Base Queries

4 Frequently Asked Questions (FAQ)

1 Introduction to RKNN3 MCP

RKNN3 MCP is a model deployment workflow assistant for Rockchip RKNN3 platforms. It helps AI agents organize model analysis, ONNX export, RKNN3 conversion, accuracy validation, and on-device deployment into traceable, resumable tasks. Operations are executed in the client environment, as illustrated below:

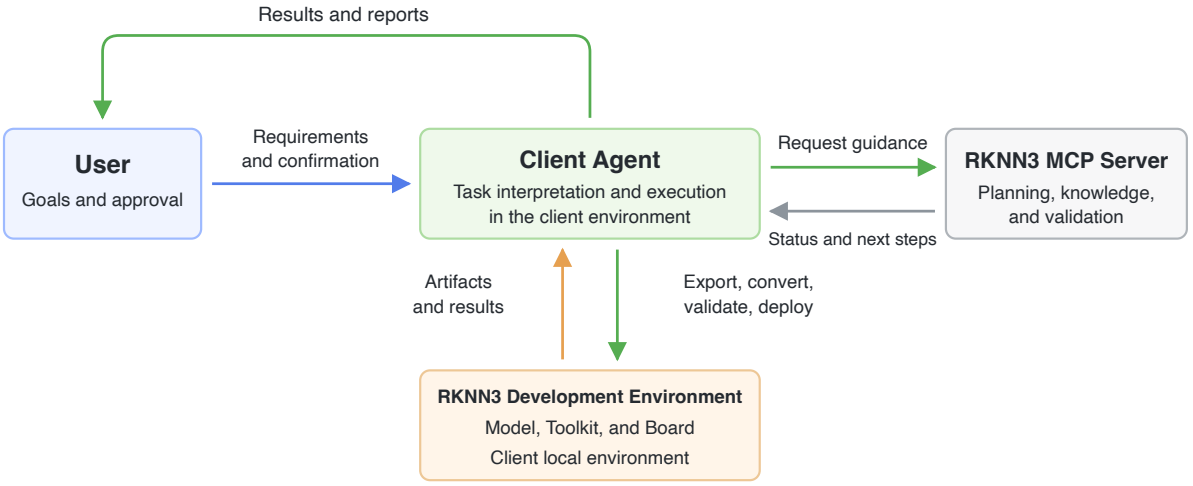


Figure 1-1 RKNN3 MCP Collaboration Model

The MCP Server handles deployment planning, knowledge provision, parameter validation, and task state recovery. The agent performs all operations involving models, scripts, RKNN3 Toolkit, and development boards in the client environment. The MCP Server does not read client files or execute client programs. It does not run Python, RKNN3 Toolkit, adb, or on-device programs, and does not connect directly to development boards.

The current workflow covers CNN, ViT, LLM, VLM, ASR, and other model types. Supported target platforms include RK1820, RK1828, and RK3572. Features include LoRA conversion and deployment, as well as multi-card deployment and inference. MCP analyzes deployment requirements and orchestrates the workflow based on the model and target platform. For unsupported cases, it provides reasons and suggested actions.

2 Quick Start

2.1 Before You Begin

Before using RKNN3 MCP, prepare the model weights and an accessible working directory.

If you need to use RKNN3 Toolkit, refer to the following development environment configurations:

Operating system version	**Ubuntu 22.04 (x64)**	**Ubuntu 24.04 (x64)**	**macOS 13+ (Apple Silicon)**
Python version	3.10	3.12	3.10 / 3.12

The macOS (Apple Silicon) version of RKNN3 Toolkit supports only offline conversion, build, and export of ONNX and existing LLM models. Board runtime, device discovery, NTP/ADB transport, performance evaluation, and memory evaluation are not supported. Wheels for Intel Mac or Universal2 are not provided.

To reduce time spent waiting for downloads during a task, we recommend preparing the required components below in advance and providing their local paths and versions to the agent:

- **RKNN3 Toolkit**: Select the wheel and accompanying requirements from the same release directory according to the host operating system, architecture, and Python ABI (such as `cp310`).
  Download directory: https://github.com/airockchip/rknn3-toolkit/tree/main/rknn3-toolkit
- **RKNN3 Runtime**: For on-device deployment, prepare the runtime library, matching backend libraries, and communication services. Match them to the target platform, board operating system, architecture, and deployment mode.
  Download directory: https://github.com/airockchip/rknn3-toolkit/tree/main/rknn3-runtime
- **RKNN3 Model Zoo**: Select export, conversion, and deployment examples for the model and SDK version. You only need to download the files required for the task.
  Repository: https://github.com/airockchip/rknn3-model-zoo
- **Firmware**: Select firmware that matches the board model, hardware revision, and deployment mode. RK1820 / RK1828 firmware is available at the link below. For other boards, use the board vendor's official release channel.
  RK1820 / RK1828 firmware: https://console.box.lenovo.com/l/PHk0PF

For tasks that require a development board, check the firmware, RKNN3 Toolkit, and Runtime before starting. We recommend using the matching component versions from the same SDK release package. Refer to the official release notes for specific version compatibility.

If you need to download models, obtain reference code, or install dependencies, ensure that the client's network or proxy can access GitHub, Python package repositories, and other required sources.

2.2 Connecting to RKNN3 MCP

2.2.1 Connection Requirements

Choose an AI development tool that supports remote MCP tool calls (hereafter referred to as the "client"). For model conversion and on-device deployment, the agent must be able to read and write the working directory, execute commands, and access the required Python environment and RKNN3 components. For on-device operations, it must also be able to connect to the development board.

For client installation, sign-in, and model configuration, refer to the client's official documentation.

2.2.2 Connection Configuration

Setting	Value
Server name	`rknn3-mcp`
Server URL	https://rknn-mcp.rock-chips.com/mcp
Transport	Streamable HTTP (shown as HTTP in some clients)

1. Open the client's MCP configuration settings and add a remote MCP server.
2. Enter the server name, URL, and transport from the table above, then save the configuration.
3. Restart the client or start a new session as required by the client to apply the configuration. Confirm that the server and its tools are enabled.

Menu names and configuration formats may vary by client. Refer to the client's MCP connection instructions.

2.2.3 Verifying the Connection

Once the configuration takes effect, confirm in the client's MCP management interface that the server is enabled and its tools are loaded. Then send the following prompt to the agent:

```
Check whether rknn3-mcp is connected and whether its tools are available.
Only verify the connection. Do not create a deployment task or convert a model.
```

After confirming that the server is connected and its tools are loaded and available to the agent, proceed to the deployment examples in Section 2.3. If the connection fails or tools are unavailable, first check the server URL, network connectivity, and whether the tools are enabled.

2.3 Deployment Examples

When using the following templates, replace placeholders such as `<MODEL_PATH>` with actual values. Keep or remove optional fields as needed. For the Python environment, specify either `uv:<VENV_PATH>` or `conda:<ENV_NAME>`.

2.3.1 Basic Example

```
Model weights: <MODEL_PATH>
Working directory: <WORKSPACE_PATH>
Python environment (optional): uv:<VENV_PATH> or conda:<ENV_NAME>
Target platform: RK1828

Use rknn3-mcp to convert the model to RKNN3.
Perform conversion only. Do not deploy to a board or run full accuracy analysis.
If rknn3-mcp is disconnected or unavailable, tell me first.
Do not bypass MCP to perform the task independently.
```

After receiving the prompt, the agent obtains guidance from RKNN3 MCP, analyzes the model structure and deployment requirements, and proposes a deployment plan for the target platform.

An example deployment plan is shown below (excerpt). The values are illustrative; review the plan returned for your own model.

Example Deployment Plan (Excerpt)

Parameter	Proposed Value
Model segment	`qwen3_5_text`
ONNX export	`mode=fp32`, standard export without GRQ
RKNN quantization	`dtype=w4a16`, `quant=true`: 4-bit weights and 16-bit activations
Algorithm and granularity	`normal` / `group32`: groups of 32 weights
Alternative quantization types	`w4a16`, `w6a16`, `w8a16`, `w16a16`, `w8a8`; higher weight bit widths increase weight memory usage
Input sequence lengths (`Seq_lens`)	`[1, 128]` tokens: 1 is typically used for token generation and 128 for block input processing; usage depends on the inference program
KV Cache capacity and storage type	2048 tokens; Float16
Paged Attention	Enabled; 512 tokens per page; one request (single-request FullAttention)
Page allocation	Initially 512 tokens (one page), with additional pages allocated as needed
Session context limit	2048 tokens

Before confirming the plan, check the following:

- **Target and stage scope**: Target platform, execution stages, stopping point, and whether on-device deployment and accuracy analysis are included.
- **Model segments and inputs**: Segments or subgraphs to convert, input dimensions, and dynamic shape settings.
- **Quantization and calibration materials**: Quantization method for each segment, required calibration data or configuration paths, and preprocessing requirements.
- **Execution environment**: Working directory, Python environment, and the paths and versions of Toolkit and Model Zoo.

- **Board and runtime parameters (when a board is required)**: Board model, connection method, Runtime version, and inference parameters.

Once the plan is correct, reply "Accept" to continue. If changes are needed, request them first, for example: "Change the quantization type to `w8a16`." Review and confirm the updated plan after the agent presents it.

2.3.2 Specifying Deployment Stages

With a Development Board: ADB Deployment

The following template connects to the development board using ADB and completes deployment without on-device accuracy analysis. For the ADB device identifier, enter the USB device serial number or the network address (`IP:port`, for example, `192.168.1.100:5555`).

When using network ADB, the agent follows MCP guidance to establish the connection before performing subsequent operations.

```
Model weights: <MODEL_PATH>
Working directory: <WORKSPACE_PATH>
Python environment (optional): uv:<VENV_PATH> or conda:<ENV_NAME>
Target platform: RK1828
Connection method: ADB
ADB device identifier: <ADB_SERIAL>
GCC path: <GCC_PATH>
Model Zoo (optional): <MODEL_ZOO_PATH>

Use rknn3-mcp to deploy the model with RKNN3.
Do not perform on-device accuracy analysis.
If rknn3-mcp is disconnected or unavailable, tell me first.
Do not bypass MCP to perform the task independently.
```

On-device accuracy analysis takes longer and consumes more agent tokens. If you only need to verify that the model can be converted to RKNN3, use the basic example in Section 2.3.1.

Without a Development Board: Simulator Accuracy Analysis

```
Model weights: <MODEL_PATH>
Working directory: <WORKSPACE_PATH>
Python environment (optional): uv:<VENV_PATH> or conda:<ENV_NAME>
Model Zoo (optional): <MODEL_ZOO_PATH>

Use rknn3-mcp to perform RKNN3 simulator accuracy analysis for the model.
If rknn3-mcp is disconnected or unavailable, tell me first.
Do not bypass MCP to perform the task independently.
```

2.3.3 Specifying Features

Using LoRA

To use LoRA, provide the base model and its corresponding LoRA directory. The directory should contain the adapter configuration file `adapter_config.json`, the weight file `adapter_model.safetensors`, and any other required supporting files.

```
Model weights: <MODEL_PATH>
LoRA directory: <LORA_PATH>
Working directory: <WORKSPACE_PATH>
Python environment (optional): uv:<VENV_PATH> or conda:<ENV_NAME>
Target platform: RK1828
Connection method: ADB
ADB device identifier: <ADB_SERIAL>
GCC path: <GCC_PATH>
Model Zoo (optional): <MODEL_ZOO_PATH>

Use rknn3-mcp to deploy the model with RKNN3 using the LoRA adapter above.
Do not perform on-device accuracy analysis.
If rknn3-mcp is disconnected or unavailable, tell me first.
Do not bypass MCP to perform the task independently.
```

Customizing the Paged Attention Page Size

To adjust the Paged Attention page size, specify it in the prompt. The following example sets the page size to 512 tokens.

```
Model weights: <MODEL_PATH>
Working directory: <WORKSPACE_PATH>
Python environment (optional): uv:<VENV_PATH> or conda:<ENV_NAME>
Target platform: RK1828
Connection method: ADB
ADB device identifier: <ADB_SERIAL>
GCC path: <GCC_PATH>
Model Zoo (optional): <MODEL_ZOO_PATH>

Use rknn3-mcp to set the Paged Attention page size to 512 tokens
and deploy the model with RKNN3. Do not perform on-device accuracy analysis.
If rknn3-mcp is disconnected or unavailable, tell me first.
Do not bypass MCP to perform the task independently.
```

2.3.4 Minimal Deployment Prompt

```
Model weights: <MODEL_PATH>
Working directory: <WORKSPACE_PATH>

Use rknn3-mcp to get this model running on an RK1828 development board.
If rknn3-mcp is disconnected or unavailable, tell me first.
Do not bypass MCP to perform the task independently.
```

We recommend using the complete templates above to specify the model path, working directory, execution environment, board information, and deployment stages. This reduces the need for the agent to infer the environment and task objectives.

3 Knowledge Base Queries

RKNN3 MCP also provides knowledge base queries. Ask the agent questions about RKNN3 in natural language. The agent uses MCP's built-in knowledge base to return technical explanations, API information, and usage recommendations. Example topics include:

- RKNN3 Toolkit APIs and usage;
- rknn3-server configuration;
- RKNN3 Runtime APIs and operating mechanisms;
- RKNN3Lite usage.

For example, send: "Use rknn3-mcp to explain how to use the llm_mode parameter in RKNN3Lite and provide a short Python example." An example response is shown below (excerpt).

llm_mode Usage

`llm_mode` selects the operating mode when creating an `RKNN3Lite` instance. The default is `False`:

- `llm_mode=False` (default): Conventional models use `load_rknn → init_runtime → inference()`.
- `llm_mode=True`: LLM/VLM models use LLM sessions for streaming generation. The direct initialization flow is `load_rknn → init_runtime(llm_args, llm_callback) → session_run()`.

Set `llm_mode=True` for LLM/VLM models; keep `False` for conventional models.

Short Example

```
from rknn3lite.api.rknn3_lite import RKNN3Lite

# Assume img and callback are prepared for the selected model.
# Conventional model: llm_mode defaults to False
lite = RKNN3Lite(verbose=True)
lite.load_rknn(model_path='model.rknn', weight_path='model.weight')
lite.init_runtime(target='rk1820', core_mask=0x01)
out = lite.inference(inputs=[img], data_format=['nhwc'])

# LLM model: llm_mode=True is required
llm = RKNN3Lite(llm_mode=True, verbose=True)
llm.load_rknn(model_path='Qwen3.rknn', weight_path='Qwen3.weight')
ARGS = [{"max_new_tokens": 256, "top_k": 1,
         "repeat_penalty": 1.1, "special_eos_id": 151645}]
llm.init_runtime(target='rk1820', core_mask=0xff,
                 llm_args=ARGS, llm_callback=callback)
ret, [n_dec, n_pre, t0, t1] = llm.session_run(prompt="Introduce RKNN3")
```

4 Frequently Asked Questions (FAQ)

1. Is a network connection required to use the MCP Server?

Yes. The client needs a network connection to access the RKNN3 MCP service.

2. Does the MCP Server upload my model or operate the development board directly?

No. Model weights, source code, and RKNN3 Toolkit remain in your local machine or development server environment. The agent performs model processing and board operations in that environment. The MCP Server provides deployment guidance, maintains the task workflow, and validates task results. It does not read client files or execute client programs.

3. Why did the agent not call RKNN3 MCP?

First, confirm that MCP is connected and authorized for use. Then explicitly include "Use rknn3-mcp" in your prompt. If the agent still does not call it, ask the agent to explain why MCP is unavailable instead of substituting other sources directly.

4. Can every model be converted to RKNN3?

Conversion is not guaranteed. The outcome depends on the model structure, ONNX graph, RKNN3 Toolkit support, quantization scheme, and target board environment. RKNN3 MCP generates a deployment plan first and continues execution after user confirmation.

5. When do I need to provide development board information?

Provide it only for on-device accuracy analysis, on-device inference, or full deployment. Board connection information is not required for model analysis, ONNX export, or RKNN3 conversion alone.